

Dipole Antenna Simulation using OpenEMS (Python) – Results

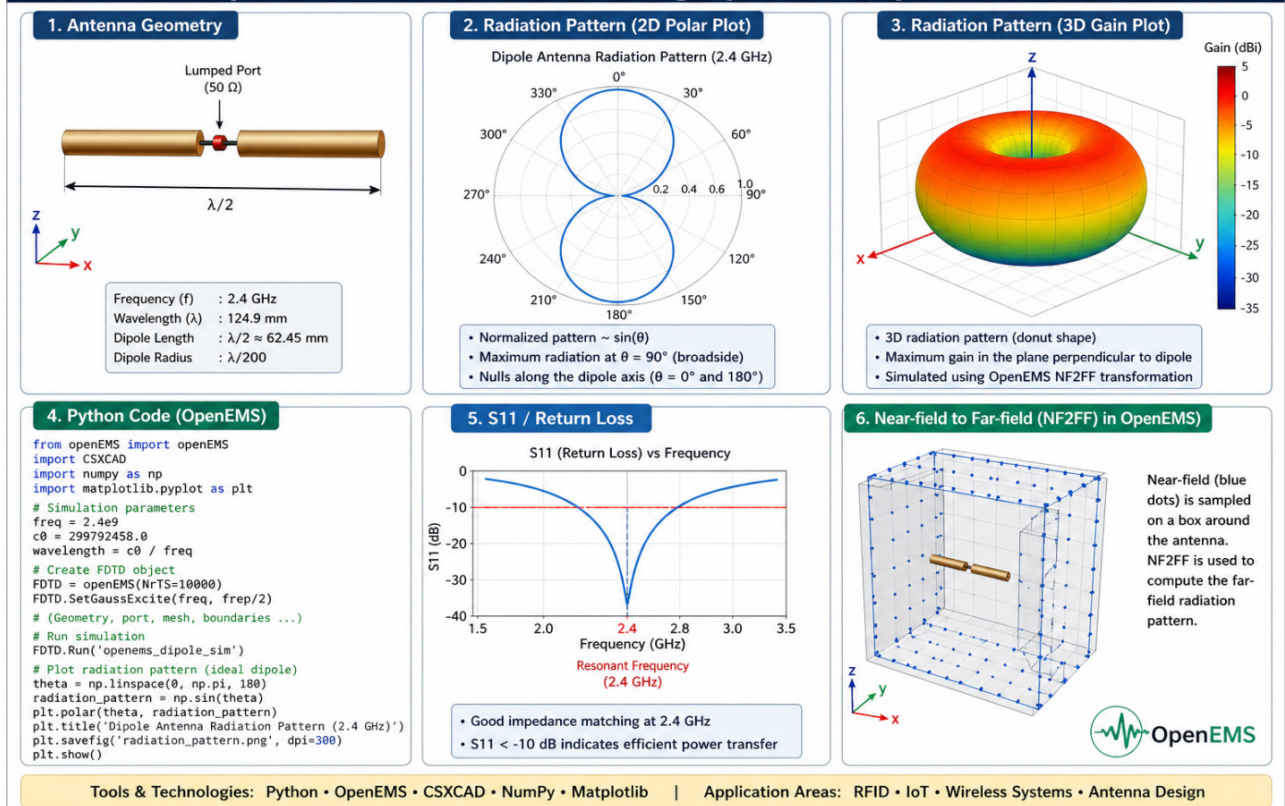


Figure 2: RF antenna radiation using Python on openEMS

```
stop2t = [dipolelength/2, 0, 0]

CSXt.AddMetal("dipole")
CSXt.AddCylinder("dipole", 0, start1t, stop1t, dipoleradius)
CSXt.AddCylinder("dipole", 0, start2t, stop2t, dipoleradius)

# Add excitation port

tport = FDTD.AddLumpedPort(1, 50, start1t, stop1t, [0, 0, 1],
True)

# Boundary conditions
FDTDt.SetBoundaryCond(['PML_8'] * 6)

# Run simulation
simpath = "openemsdipolesim"
if not os.path.exists(simpath):

    os.mkdir(simpath)

CSX_file = os.path.join(sim_path, "dipole.xml")
CSXt.Write2XML(CSX_file)

FDTDt.Run(sim_path)
```

```
# Post-processing (simplified radiation pattern)
theta2 = np.linspace(0, np.pi, 180)
radiation_pattern2 = np.sin(theta2) # ideal dipole
approximation

# Radiation Analytics
plt.figure()
plt.polar(theta, radiation_pattern)
plt.title("Dipole Antenna Radiation Pattern (2.4 GHz)")
plt.savefig(os.path.join(sim_path, "radiationpattern.png"))
plt.show()
```

Researchers and telecom engineers can simulate wireless and telecom scenarios using openEMS and the Python interfacing library for high performance applications before deploying the devices and modules on the physical location. Radiations, signals and associated parameters can be evaluated for error-free implementations. **END** 🐧

By: Dr Gaurav Kumar

The author is associated with various academic and research institutes for delivering expert lectures and conducting technical workshops on the latest technologies and tools.